Data Structure And Algorithm

Name - Shivam Kumar
Rollno. - 63

# Submission of Introduction

Problem 1.

(a)

| | Algorithm A | Algorithm B. |
|---|---|---|
| Input | Yes | Yes |
| Output | Yes (The avg) | Yes (A calculated value) |
| Definiteness | Yes. Steps are clear and unbiguous | Yes. Steps are clear. |
| Finiteness | Yes. Loop runs n times | Yes. Loop runs n-1 and n times respect. |
| Effectiveness. | Yes. Operation are basic and doable. | No, while the steps are doable they do not effectively solve the problem of finding an avg. |

(b) Step-by-Step Execution -

Input = temperature = [20, 25, 22, 24, 25], n = 5.

Algorithm A

1. total = 0
2. i = 1 : total = 0+20 = 20
3. i = 2 : total = 20+25 = 45
4. i = 3 : total = 45+22 = 67
5. i = 4 : total = 67+24 = 91
6. i = 5 : total = 91+21 = 112
7. average = 112/5 = 22.4
8. Result : 22.4

Algorithm B.

1. Sum = 20
2. : i = 2 to 5 : 20+25+22+24+21 = 112
3. : i = 1 to 5
   - i = 1 : sum = 112/5 = 22.4
   - i = 2 : Sum = 22.4/5 = 4.48
   - i = 3 : Sum = 4.48/5 = 0.896
   - i = 4 : Sum = 0.896/5 = 0.1792
   - i = 5 : sum = 0.1792/5 = 0.03584
4. Result = 0.03584 (Incorrect)

(c) Algorithm B fails because it nests the division operation inside a loop

(d) Comparison

Simplicity:- Algorithm A is simpler and follow the standard mathematical definition

Correctness - Algorithm A is correct.

Efficiency - Algorithm A uses a loop and one division.

Recommendation - Algorithm A must be chosen for prediction.

Ques 2 (a) Worst case Operation (n = 10,000000)

1. Algorithm A (Linear Search) - In the worst case, the target is very last item or not there at all.

→ cal = n operations

→ Total = 10000000 comparisons.

2. Algorithm B (Binary Search) - This repeatedly halving the search space.

Calculation = ($\log_2 n$)

Total = 24 comparision.

3. Algorithm C (Hash Table) - A well defined hash table provides a direct backup.

Cal - O(1)

Total = 1 comparison.

(b) Big-O Notation & Justification

Algorithm A - O(n) (Linear Time)

The no. of operations grows in direct proportion to the input size.

Algorithm B - O(logn) (Logarithmic Time)

Each comparison eliminates half of the remaining elements.

Algorithm C - O(1) (Constant Time)

The algorithm uses a hash fxn to compute an index directly.

(d) Peak load Analysis (50000 check/sec)

Algorithm A - 50000 X 10000000 = 500000000000 ns (500 sec)

It takes over 8 min to process 1 sec of traffic.

Algorithm B - 50000 X 24 ns = 1200000 ns (0.0012 sec)

It uses only 0.12 % of the available second.

Algorithm C - 50000 X/m = 50000 ns (0.00005 ms)

It is nearly instantaneous.

(e) The 'worth it'

To find when sorting and Binary search better than repeated linear searches.

$K \cdot n > (n \log n) + (k \cdot \log n)$

$K \cdot n = n \log n \Rightarrow K \geq \log n$.

Ques 3. (a) Time complexity Analysis

A: Nested loop

1. The outer loop runs m times.
2. For each prerequisite, the inner loop searches the complete list, which has n elements
3. In worst case, it performs n comparisons for each of the m prerequisites.
4. Result = $O(m \times n)$

B: Hash Set -

1. Building the Hash set involves iterating through n complete courses and inserting each. Total : $O(n)$
2. Checking the prerequisite involves a single loop of m iterations. Total $O(m)$
3. Result : $O(n+m)$.

(b) Space complexity :-

A- Result $O(1)$ constant space)

B- Result : $O(n)$ (Linear search relative to the no. of completed courses).

(c) Practical comparison (A)

1. Total comparison (A).
   - per student : $m \times n = 5 \times 40 = 200$ comparison.
   - Per day : 100000 students $\times 200 = 20000000$ comparison / day.
2. Total memory Usage (B) -
   - Per Student $n \times 8$ bytes. $40 \times 8 = 320$ bytes.
   - Per day. $100000 \times 320$ bytes $= 32000000$ bytes $= 32$ MB.

(c) Hybrid Approach & Threshold

- If m is small: Use A.
- If m is large: Use B.

Determine the threshold

$$m \cdot n > (n + m)$$

$n = 40$

$$40m > (40 + m)$$

$$39m > 40$$

$$\boxed{m > 1.025}$$

The switch happen when the cost of the nested loop exceeds the cost of building the hash set.

Que 4: (a) ~~Exam~~ Formal Proof of worst case complexity.

To prove. $f(n) = O(g(n))$.

1. Bubble sort: $O(n^2)$

Fxn:- $f(n) = \frac{n(n-1)}{2} = 0.5n^2 - 0.5n$

Proof:- $0.5n^2 - 0.5n \leq c_1 n^2$

If we choose $c = 1$, then $0.5n^2 - 0.5n \leq n^2$ is true of all $n \geq 1$

$$\boxed{\text{Constant} = c > 1 \quad n_0 = 1}$$

2. Insertion sort:-

Fxn: $f_n = \sum_{i=1}^{n-1} i = \frac{n^2 - n}{2}$

Proof. Similar to bubble sort. $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$

Constt:- $c = 0.5$, $n_0 = 1$

(b) Data Type matching

Type A
Nearly Sorted — Insertion Sort

Type B
Random — Quick Sort

Type C
Reverse Sorted — Merge Sort.

(c) $\Theta(n\log n)$ vs. $O(n\log n)$

- Merge Sort - $\Theta(n\log n)$ : The $\Theta$ (Big theta) notation represents tight bound.
- Quick Sort - $O(n\log n)$ : - (Average) For Quick sort, $n\log n$ is a upper bound for the average case, but it is not a tight bound for all cases.
- Difference - $\Theta$ implies the algorithm performance is locked into growth rate
  $O$ provides a ceiling but allows for the possibilities that the algorithm could be faster.

(e) Mystery Sort vs. Merge Sort

Mystery Sort : $T_M(n) = 5n^2 + 100n + 1000$
Merge Sort : $T_{MS}(n) = 10n\log_2 n$

By setting $5n^2 + 100n + 1000 = 10n\log_2 n$ and solve for $n$:

→ For very small $n$ ($n < 5$), Mystery sort's overhead a high.
→ In the range $n \approx 10$ to $50$, Mystery sort might appear competitive.
→ Result : For an $n > 100$ Merge sort will be significantly faster.